Objective: Launch a Node.js application in a Docker container and connect it to My SQL Database using environment variables. AWS resources EC2 and RDS are provisioned and configured by ansible.

Complete Step-by-Step Execution Guide

What you will build	Details
Cloud Provider	AWS — us-east-1 (N. Virginia)
Total EC2 Instances	2 (1 Control Node + 1 Managed Nodes)
Config Mgmt Tool	Ansible — provisions 1 EC2 instance and RDS database
Config Mgmt Tool	Ansible — installs & starts docker and create docker container with Node.js web application on 1 Managed Node
OS on all nodes	Linux (ami-004021b186a0c56f8)
Default SSH user	Ec2-user
End result	Launch a Node.js application in a Docker container and connect it to My SQL Database using environment variables. Node.js web application should be accessible from browser using URL: http://<<NODE-IP>>

PHASE 0: PRE-REQUISITES

What you need before starting

AWS Setup — Create a Key Pair

You need ONE. pem key pair for all 2 EC2 instances. Do this first.

🔑 **AWS Console → EC2 → Key Pairs → Create key pair**

Name it something memorable, e.g.: mylab-key

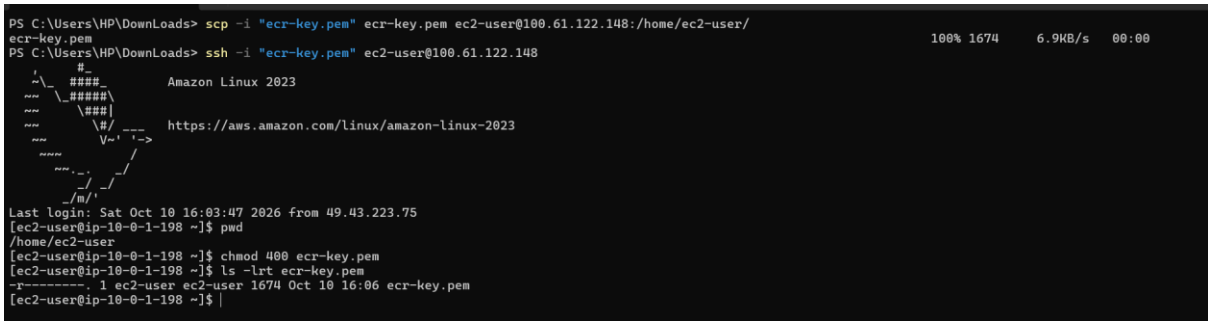
Format: .pem (for Linux/Mac/Git Bash)

Download it → it saves to your Downloads folder as mylab-key.pem

IMPORTANT: You can only download it once. Keep it safe.

PHASE 1: SSH KEY SETUP

Transfer your .pem key to the Control Node



What is SCP?

SCP = Secure Copy Protocol. It transfers files over SSH.

-i mylab-key.pem → use this key to authenticate with the remote server

ubuntu@<IP>:/home/ec2-user/ → destination path on the Control Node

PHASE 2: INSTALL ANSIBLE

Set up Ansible on the Control Node

STEP 1 Set Key Permissions on Control Node

The .pem key you just uploaded needs strict permissions:

```
[ec2-user@ip-10-0-1-198 ~]$ pwd
/home/ec2-user
[ec2-user@ip-10-0-1-198 ~]$ chmod 400 ecr-key.pem
[ec2-user@ip-10-0-1-198 ~]$ ls -lrt ecr-key.pem
-r--------. 1 ec2-user ec2-user 1674 Oct 10 16:06 ecr-key.pem
[ec2-user@ip-10-0-1-198 ~]$
```

STEP 2 Update System & Install docker

```
Last login: Sat Oct 10 16:06:51 2026 from 49.43.223.75
[ec2-user@ip-10-0-1-198 ~]$ sudo yum update -y
Last metadata expiration check: 20:54:31 ago on Fri Oct  9 19:20:37 2026.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-10-0-1-198 ~]$ sudo yum install ansible -y
Last metadata expiration check: 20:54:44 ago on Fri Oct  9 19:20:37 2026.
Package ansible-8.3.0-1.amzn2023.0.3.noarch is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-10-0-1-198 ~]$ ansible --version
ansible [core 2.15.3]
  config file = None
  configured module search path = ['/home/ec2-user/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3.9/site-packages/ansible
  ansible collection location = /home/ec2-user/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.9.25 (main, Jul 15 2026, 00:00:00) [GCC 11.5.0 20240719 (Red Hat 11.5.0-5)] (/usr/bin/python3.9)
  jinja version = 3.1.4
  libyaml = True
[ec2-user@ip-10-0-1-198 ~]$
```

STEP 3 Install boto3 botocore

```
[ec2-user@ip-10-0-1-198 ~]$ ansible-galaxy collection install amazon.aws community.docker
Starting galaxy collection install process
Nothing to do. All requested collections are already installed. If you want to reinstall them, consider using `--force`.
[ec2-user@ip-10-0-1-198 ~]$ sudo yum install awscli
Last metadata expiration check: 20:57:40 ago on Fri Oct  9 19:20:37 2026.
Package awscli-2-2.36.47-4.amzn2023.0.1.noarch is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-10-0-1-198 ~]$ sudo yum install python3-pip -y
Last metadata expiration check: 20:57:59 ago on Fri Oct  9 19:20:37 2026.
Package python3-pip-21.3.1-2.amzn2023.0.21.noarch is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-10-0-1-198 ~]$ pip install boto3 botocore
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: boto3 in ./.local/lib/python3.9/site-packages (1.42.97)
Requirement already satisfied: botocore in ./.local/lib/python3.9/site-packages (1.42.97)
Requirement already satisfied: s3transfer<0.17.0,>=0.16.0 in ./.local/lib/python3.9/site-packages (from boto3) (0.16.1)
Requirement already satisfied: jmespath<2.0.0,>=0.7.1 in /usr/lib/python3.9/site-packages (from boto3) (0.10.0)
Requirement already satisfied: urllib3<1.27,>=1.25.4 in /usr/lib/python3.9/site-packages (from botocore) (1.25.10)
Requirement already satisfied: python-dateutil<3.0.0,>=2.1 in /usr/lib/python3.9/site-packages (from botocore) (2.8.1)
Requirement already satisfied: six>=1.5 in /usr/lib/python3.9/site-packages (from python-dateutil<3.0.0,>=2.1->botocore) (1.15.0)
[ec2-user@ip-10-0-1-198 ~]$
```

PHASE 3: SSH AGENT SETUP

Load your key into memory & test node connections

```
[ec2-user@ip-10-0-1-198 ~]$ sudo systemctl status sshd
● sshd.service - OpenSSH server daemon
     Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-10-09 19:13:28 UTC; 21h ago
       Docs: man:sshd(8)
             man:sshd_config(5)
   Main PID: 2556 (sshd)
      Tasks: 1 (limit: 4659)
     Memory: 12.3M
        CPU: 2.664s
     CGroup: /system.slice/sshd.service
             └─2556 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Oct 10 16:03:46 ip-10-0-1-198.ec2.internal sshd-session[67497]: pam_unix(sshd:session): session opened for user ec2-user(uid=1000) by (uid=0)
Oct 10 16:05:43 ip-10-0-1-198.ec2.internal sshd-session[67652]: Connection reset by 205.210.31.225 port 61976 [preauth]
Oct 10 16:06:16 ip-10-0-1-198.ec2.internal sshd-session[67654]: Accepted publickey for ec2-user from 49.43.223.75 port 55777 ssh2: RSA SHA256:fDWmQz4hqvRfp
Oct 10 16:06:17 ip-10-0-1-198.ec2.internal sshd-session[67654]: pam_unix(sshd:session): session opened for user ec2-user(uid=1000) by (uid=0)
Oct 10 16:06:50 ip-10-0-1-198.ec2.internal sshd-session[67734]: Accepted publickey for ec2-user from 49.43.223.75 port 65467 ssh2: RSA SHA256:fDWmQz4hqvRfp
Oct 10 16:06:50 ip-10-0-1-198.ec2.internal sshd-session[67734]: pam_unix(sshd:session): session opened for user ec2-user(uid=1000) by (uid=0)
Oct 10 16:14:56 ip-10-0-1-198.ec2.internal sshd-session[68063]: Accepted publickey for ec2-user from 49.43.223.75 port 55943 ssh2: RSA SHA256:fDWmQz4hqvRfp
Oct 10 16:14:56 ip-10-0-1-198.ec2.internal sshd-session[68063]: pam_unix(sshd:session): session opened for user ec2-user(uid=1000) by (uid=0)
Oct 10 16:26:23 ip-10-0-1-198.ec2.internal sshd-session[68533]: Accepted publickey for ec2-user from 49.43.223.75 port 63278 ssh2: RSA SHA256:fDWmQz4hqvRfp
Oct 10 16:26:23 ip-10-0-1-198.ec2.internal sshd-session[68533]: pam_unix(sshd:session): session opened for user ec2-user(uid=1000) by (uid=0)
[ec2-user@ip-10-0-1-198 ~]$ eval $(ssh-agent -s)
Agent pid 68575
[ec2-user@ip-10-0-1-198 ~]$ pwd
/home/ec2-user
[ec2-user@ip-10-0-1-198 ~]$ ssh-add ecr-key.pem
Identity added: ecr-key.pem (ecr-key.pem)
[ec2-user@ip-10-0-1-198 ~]$ ssh-add -l
2048 SHA256:fDWmQz4hqvRfp/qzxtbRkyvet6ROhyPNn1YvOA8dDB4 ecr-key.pem (RSA)
[ec2-user@ip-10-0-1-198 ~]$
```

STEP 4 Test SSH Connections to Each Managed Node

Now test that you can reach each managed node. Use PRIVATE IPs of node2:

```
[ec2-user@ip-10-0-1-198 ~]$ ssh ec2-user@10.0.1.112

A newer release of "Amazon Linux" is available.
  Version 2023.10.20260105:
```

STEP 5 Configure AWS CLI

Open Terminal / Git Bash and configure your AWS credentials:

```
$ aws configure



# You will be prompted for 4 values:

# AWS Access Key ID     → paste your IAM Access Key
```

```
# AWS Secret Access Key → paste your IAM Secret Key

# Default region name   → us-east-1

# Default output format → json
```

💡 **Where to get Access Key & Secret Key:**

AWS Console → IAM → Users → Your User → Security Credentials → Create Access Key

PHASE 4: Run the playbook

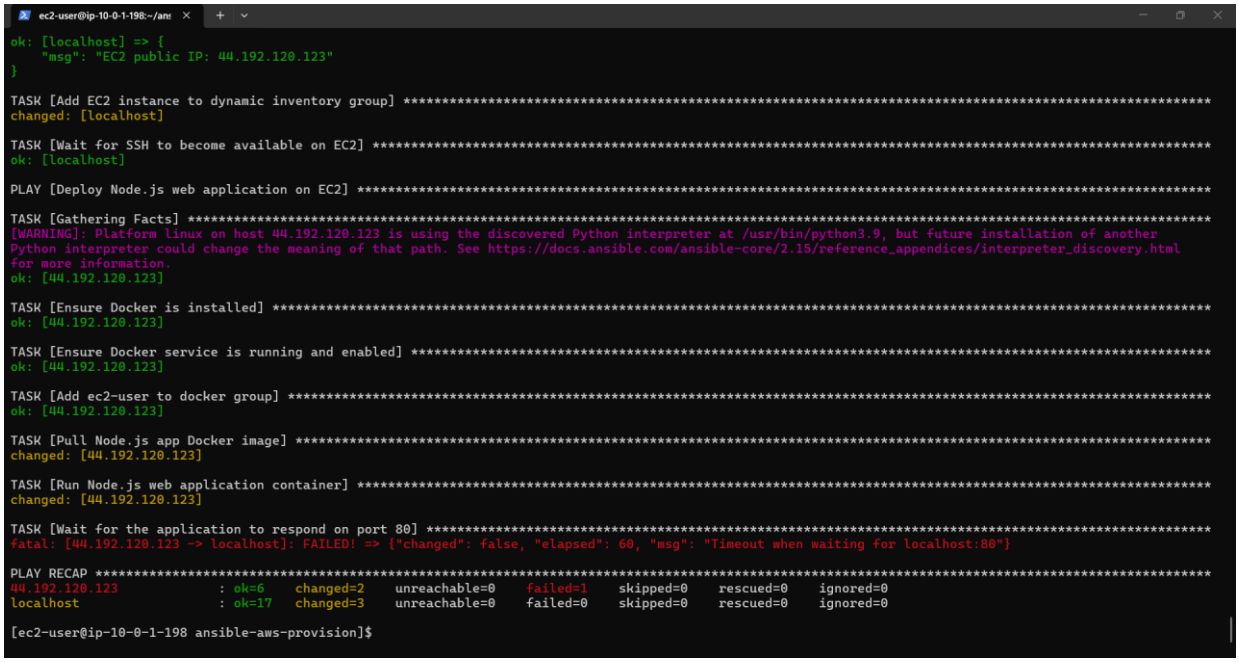
1. Set your RDS password securely via environment variable

   export RDS_PASSWORD="yourPASSPHRASE32#"

2. Run the playbook

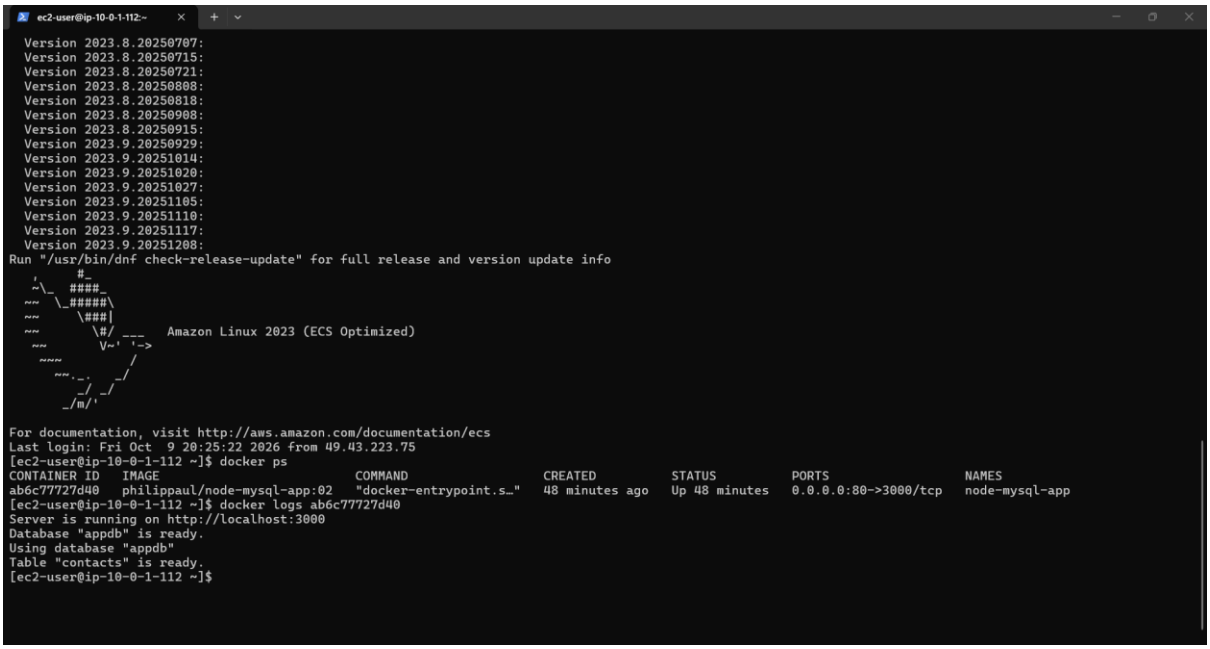
   ansible-playbook site.yml

3. Ansible playbook log

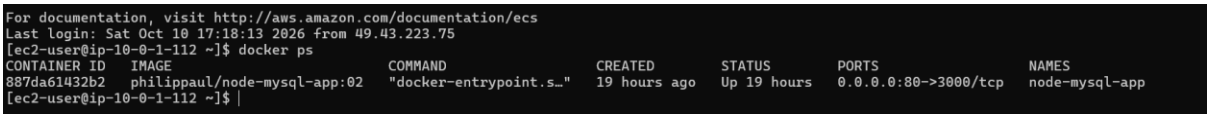


PHASE 5: VERIFY — CONFIRM Node.js application in Docker container IS RUNNING

1. Docker logs:



2. Port Mapping



PORTS:

0.0.0.0:80->3000/tcp

Node.js application listens on port 3000 inside the container.

We expose it through port 80 on browser.

Access Node.js web application from browser

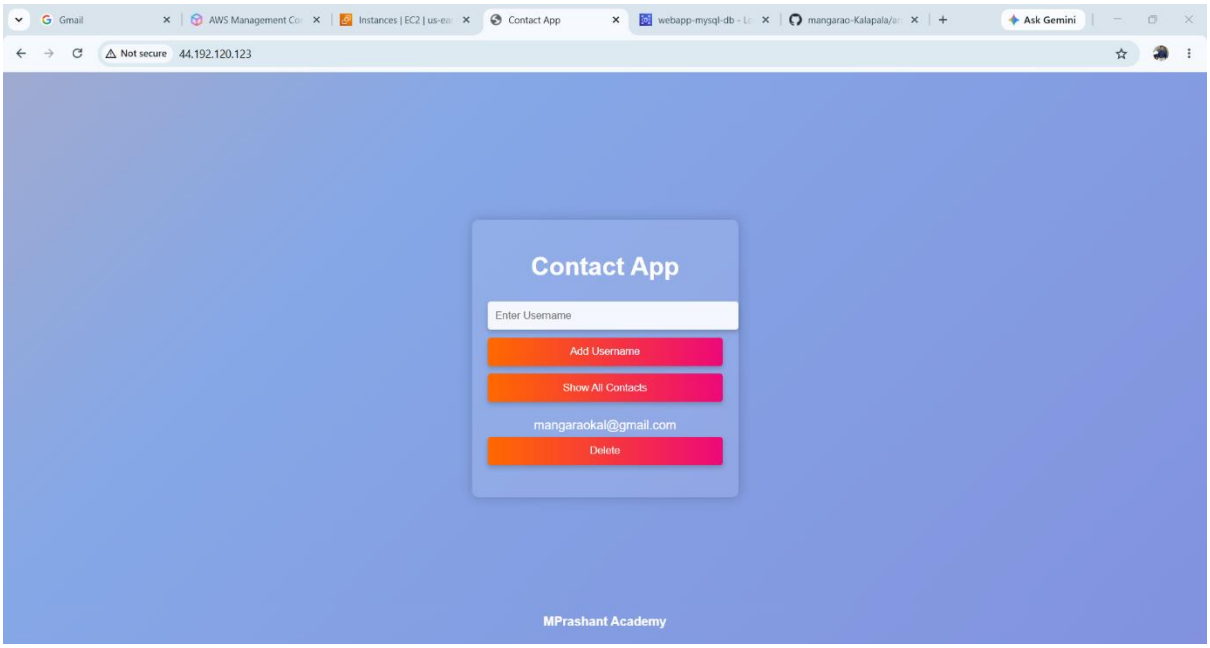
http://<<NODE-IP>>:80

or

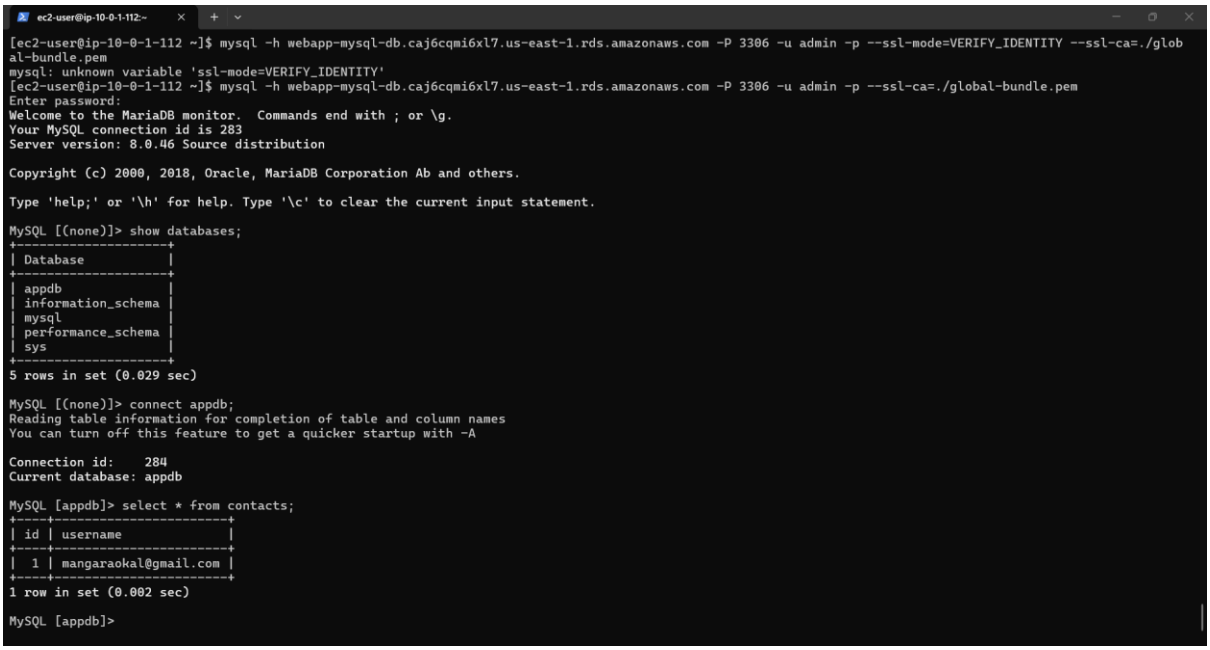
http://<<NODE-IP>>

3. Access web application from browser

http://<<NODE-IP>>



4. DB access through MySQL client:



PHASE 6: CLEANUP (Important!)

Destroy resources to avoid AWS charges

docker stop CONTAINER ID

docker rm CONTAINER ID

```
57  docker stop ab6c77727d40
58  docker rm ab6c77727d40
```

a. Delete RDS database using Ansible playbook

Change state to absent and run playbook to delete RDS database and EC2 instance using Ansible.

Example:

tasks:

- name: Delete RDS database

  amazon.aws.rds_instance:

    name: webapp-mysql-db

    state: absent

b. Delete EC2 instance using Ansible playbook

tasks:

- name: Delete EC2 instance

  amazon.aws.ec2_instance:

    name: web-app-server

    state: absent

Troubleshooting — Common Issues

Error / Problem	Fix
TASK [Create RDS DB subnet group] fatal: [localhost]: FAILED! => {"changed": false, "msg": "missing required arguments: state"}	Added below code snippet in the ansible playbook to resolve **state: present** tags: Name: webapp-db-subnet-group
The error was: botocore.exceptions.ClientError: An error occurred (InvalidAMIID.NotFound) when calling the RunInstances operation: The image id '[ami-0d3d85815a9746bc5]' does not exist	Resolution: ec2_ami_id: "ami-0d3d85815a9746bc5" it seems this is not available in us-east-1 Changed to: “ami-004021b186a0c56f8”

The key pair 'ansible-key' does not exist fatal: [44.192.120.123]: UNREACHABLE! => {"changed": false, "msg": "Failed to connect to the host via ssh: Warning: Permanently added '44.192.120.123' (ED25519) to the list of known hosts.\r\nno such identity: /home/ec2-user/.ssh/ecr-key.pem: No such file or directory\r\nec2-user@44.192.120.123: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).", "unreachable": true}	resolution 3: it seems ansible-key is in us-east-2c. so changed to ecr-key as it is available in us-east-1 Resolution: SSH AGENT SETUP has been done to resolve. sudo systemctl status sshd eval $(ssh-agent -s) ssh-add ecr-key.pem ssh-add -l